

SVP RC1 Implementation Roadmap

From spec repository to first 30-second video package

Strategy

Build the validator and package infrastructure first, then the reader and inspector, then media/audio/vision processing, then final package assembly and a real 30-second trial.

Phase	Name	Main output	Can parallelize?
00	Repo baseline	Clean committed RC1 repo foundation	No
01	Build system and CLI	CMake/vcpkg, C++20 targets, CLI skeleton	No
02	Validator core	Reports, registry load, ZIP layout checks	After 01
03	Validator deep checks	Schemas, SVPB, BLAKE3, SQLite logical rows	After 02
04	Fixture lab	Valid/invalid tiny SVP fixtures	After 02
05	Reader and inspector	Human-readable package inspection	After 04
06	Builder media ingest	FFmpeg probe, audio extraction, canonical timing	After 01
07	Audio transcript pipeline	VAD, words, speaker segments	After 06/08
08	Model bundle runtime	Native ONNX/model bundle verification	After 01
09	Vision observations	Shots, tracks, regions, depth, masks	After 06/08
10	Index and package writer	Relationships, embeddings, SQLite, final package	After 07/09
11	First video trial	Real 30-second video to .svp	After 10
12	Hardening	Implementation gap list and RC feedback	After 11

First Video Acceptance Target

The first real milestone is not a perfect production builder. It is a real pipeline that can process one clean 20 to 30 second video into a package that the validator and inspector can read.

```
svp build samples/dom-30s.mov --out runs/first-video-trial/dom-30s.svp
svp validate runs/first-video-trial/dom-30s.svp --json
svp inspect runs/first-video-trial/dom-30s.svp
```

Parallel Work

After Phase 01, validator, fixture planning, model runtime, and media ingest can proceed in parallel. After media ingest and model runtime exist, audio and vision can proceed in parallel. Indexing and final packaging come after audio and vision outputs exist.

Non-negotiables

- Do not start with the full builder.
- Do not claim a package is valid without validator output.
- No Python runtime for normal builder operation.
- Labels remain non-core.
- Use canonical model IDs everywhere normative.
- Signature authenticity findings do not affect Core validity by default.